

CSSE304 Exam 1 Paper Part

Please complete the following problems. If you have questions feel free to come up and ask (AJ), come to my classroom hours after this, or shoot me a message (Teams or Email works)! I will commit the solutions to this GitHub repo later tonight so you can review over the weekend. Good luck!

Problem 1

Write a function `sum-of-squared-odds` that takes in a list of integers and returns the sum of the squares of only the odd values. Use some combination of `map`, `apply`, and `filter` to complete the problem (no explicit recursion).

Ex. (`sum-of-squared-odds` '(1 2 3 4 5)) -> 35 ($1^2 + 3^2 + 5^2 = 35$)

```
(define (sum-of-squared-odds nums)
  (apply + (map (lambda (x) (* x x))
                (filter odd? nums))))
```

Know map, apply, + filter for the exam!

Problem 2

Write a function `sum-of-string-lengths` that takes in a list of values (strings and numbers) and returns the sum of the lengths of only the strings. Use some combination of `map`, `apply`, and `filter` to complete the problem (no explicit recursion).

Ex. (`sum-of-string-lengths` '("cat" 5 "a" 9 "tree")) -> 8 (`length`("cat") = 3, `length`("a") = 1, `length`("tree") = 4) ($3 + 1 + 4 = 8$)

```
(define (sum-of-string-lengths vals)
  (apply + (map length (filter string? vals)))))
```

Problem 3

Write a function pair-twister that takes two procedures as arguments and returns a new procedure. The returned procedure takes two lists of equal length and does the following:

1. Apply the first procedure to each pair of corresponding elements from the two lists.
2. Combine all of the resulting values into a single value using the second procedure.

Ex. 1:

```
(define add-and-sum (pair-twister + +))
(add-and-sum '(1 2 3) '(10 20 30))
;; -> (apply + (map + '(1 2 3) '(10 20 30)))
;; -> (apply + '(11 22 33))
;; -> 66
```

Ex. 2:

```
(define mult-and-max (pair-twister * max))
(mult-and-max '(1 2 3) '(5 2 1))
;; -> (apply max (map * '(1 2 3) '(5 2 1)))
;; -> (apply max '(5 4 3))
;; -> 5
```

Hint: think about map and apply

```
(define (pair-twister proc 1 proc 1 proc2)
  (lambda (list1 list2)
    (apply proc2 (map proc1 list1 list2)))))
```

This is a procedure that returns
a procedure, know how to play with funny
examples like this one for the exam.

Problem 4

Here is some code that uses let and lambda to show closures. Write what will be displayed by the code when it is run.

```
(define bank
  (let ((vault 500))
    (lambda ()
      (let ((withdrawals 0))
        (lambda ()
          (let ((fee 3))
            (set! vault (- vault 20))
            (set! withdrawals (+ withdrawals 1))
            (set! fee (+ fee withdrawals))
            (display (list vault withdrawals fee))
            (newline))))))))

(define teller1 (bank))
(define teller2 (bank))
(teller1)
(teller1)
(teller2)
(teller1)
```

Final:
(480 1 4)
(460 2 5)
(440 1 4)
(420 3 6)

Know how shared variables
work & how examples like
this work for the exam.

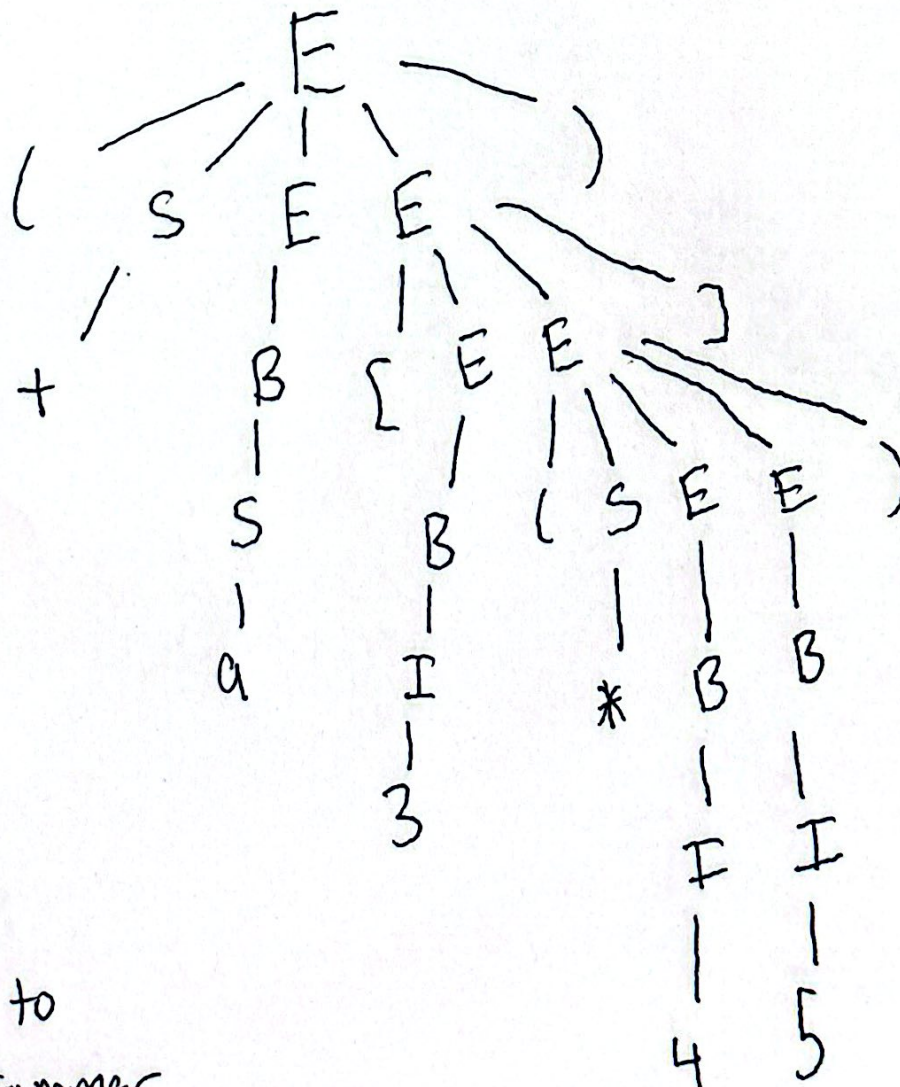
Problem 5

Use the following grammar and expression to create a grammar tree. Use E for expr, B for base val, S for symbol, and I for integer.

$\langle \text{expr} \rangle ::= (\langle \text{symbol} \rangle \langle \text{expr} \rangle \langle \text{expr} \rangle) \mid [\langle \text{expr} \rangle \langle \text{expr} \rangle] \mid \langle \text{base_val} \rangle$

$\langle \text{base_val} \rangle ::= \langle \text{integer} \rangle \mid \langle \text{symbol} \rangle$

(+ a [3 (* 4 5)])



Know how to
create a grammar
tree from a given
definition

Problem 6

Consider the following grammar:

```
<expr> ::= <num> | (<expr> <op> <expr>)  
<op> ::= + | - | * | /  
<num> ::= <integer>
```

Write "yes" next to any of the following strings that are in this grammar, write "no" otherwise:

String	Yes/No
3	<u>yes</u>
(3 + 4)	<u>yes</u>
(3 + 4 * 5)	<u>No</u>
((3 + 4) * 5)	<u>Yes</u>
(3 + (4 * 5))	<u>Yes</u>
(+ 3 4)	<u>No</u>
(3 +)	<u>No</u>
3 + 4	<u>No</u>
((3 + 4) * (5 - 6))	<u>Yes</u>
(3 ++ 4)	<u>No</u>

Know how to
determine if a
string is within
a defined grammar.